

AI Operations (AIOps) Assignment 1

Shruthi Rathod, DA24B022

Question 1 — Conceptual: Technical Debt Diagnosis

1.

- **(a) Entanglement (CACE)**

Changing the delivery time feature unexpectedly affected the favorite restaurants feature.

This is “Changing Anything Changes Everything.” A small change to one feature unexpectedly affects another part of the model’s behaviour.

- **(b) Undeclared Consumers**

The marketing team was using the model’s output without telling the ML team.

This creates a hidden dependency because the ML team doesn’t know who is using the output and that block safe changes.

- **(c) Configuration & Glue-Code Debt**

The training pipeline has 14 undocumented shell scripts and no proper system to manage them.

This makes the pipeline messy, difficult to understand, and difficult to maintain.

2.

Specific mitigation for (c) Configuration & Glue-Code Debt :

Convert the shell scripts into an MLflow Project

1. Instead of 14 separate shell scripts, we create one `MLproject` file in root of repository . This file clearly declares The conda environment , Named entry points instead of loose shell scripts ,Parameters that were previously hard-coded across scripts.
2. Each former shell script becomes a clean, documented entry point.
3. Run the entire pipeline with a single, reproducible command

This solution removes the undocumented scripts, makes the pipeline clear ,Gives a single source of truth for the training workflow and Makes the pipeline version controlled with Git.

Question 2: Written Analysis

Across the six runs, the two-layer architecture with hidden layers (128,64) and a learning rate of 0.001 produced the best performance. Runs 4 and 6 both achieved the highest validation accuracy (0.980) and near-identical F1-macro scores (0.980 and 0.979 respectively). Run 6, which paired this architecture with a larger batch size (128), is the standout result: it reached the lowest training loss of all runs (0.006) while matching Run 4’s validation accuracy.

Comparing `train_loss` to `val_accuracy`, there is mild evidence of overfitting in the four $lr = 0.001$ runs (1, 3, 4, 6): training loss falls very low (0.006–0.014) while validation loss stays noticeably higher (0.093–0.121), leaving a persistent generalization gap even though validation accuracy remains high. This gap is sharpest in Run 6, where training loss is lowest but validation loss is not the lowest of the group, suggesting the model is beginning to memorize training examples. By contrast, the $lr = 0.01$ runs (2, 5) show elevated training *and* validation loss together, which points to underfitting rather than overfitting.

Comparing hyperparameters, learning rate clearly has the larger effect on performance: every $lr = 0.01$ run underperforms (validation accuracy 0.963–0.964) regardless of architecture, while every $lr = 0.001$ run stays at 0.976–0.980. Architecture only shifts accuracy by about 0.004 at a fixed learning rate, and batch size mainly affects training time rather than accuracy.